

SIMATS ENGINEERING

ASSESSMENT TOOL 1

NAME: Y. SAI PRASANTH VARMA

REG NO: 192411191

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4308

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

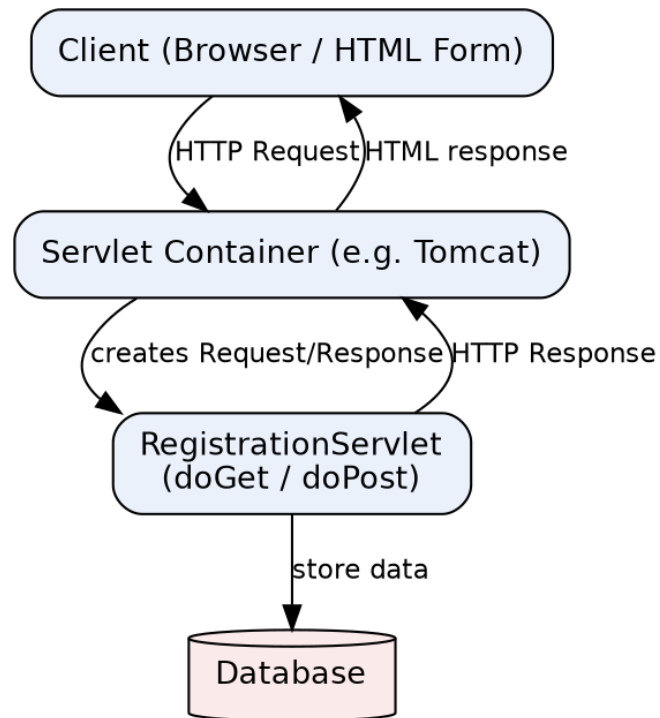
Answer:

Servlet Architecture

A servlet-based web application follows a request/response model mediated by a servlet container (e.g. Apache Tomcat):

- The browser sends an HTTP request (from the registration form) to the web server.
- The servlet container receives it, and — based on the URL pattern in web.xml or an `@WebServlet` annotation — creates `HttpServletRequest` and `HttpServletResponse` objects and dispatches them to the matching servlet instance.

- The servlet's business logic reads form fields from the request, processes/validates/stores them, and writes the response body (HTML, JSON, or a redirect).
- The container sends the response back to the browser, and keeps the servlet instance alive in memory to serve future requests (servlets are multithreaded singletons — one instance handles many requests concurrently).



HTML Form

```

<form action="register" method="post">
  <input type="text" name="name" placeholder="Full Name">
  <input type="email" name="email" placeholder="Email">
  <input type="text" name="regNo" placeholder="Registration No.">
  <input type="submit" value="Register">
</form>

```

Servlet Implementation (handles GET and POST)

```

@WebServlet("/register")
public class RegistrationServlet extends HttpServlet {

    private Connection conn; // shared DB connection/pool handle

    @Override
    public void init() throws ServletException {
        // Called once, when the container loads the servlet.
        // Ideal place to open a DB connection pool / read config.
        try {

```

```

        conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "pwd");
    } catch (SQLException e) {
        throw new ServletException("DB init failed", e);
    }
}

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    // Typically just serves the empty registration form/page.
    req.getRequestDispatcher("registerForm.jsp").forward(req, resp);
}

@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    // Handles the actual form submission.
    String name = req.getParameter("name");
    String email = req.getParameter("email");
    String regNo = req.getParameter("regNo");

    String sql = "INSERT INTO students(name,email,reg_no) VALUES (?,?,?)";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, name);
        ps.setString(2, email);
        ps.setString(3, regNo);
        ps.executeUpdate();
        resp.sendRedirect("success.html");
    } catch (SQLException e) {
        throw new ServletException("Could not save student", e);
    }
}

@Override
public void destroy() {
    // Called once, just before the container unloads the servlet
    // (e.g. on shutdown/redeploy). Release resources here.
    try { if (conn != null) conn.close(); } catch (SQLException ignored) {}
}
}

```

Servlet Life Cycle

Phase	When it runs	Purpose
-------	--------------	---------

init()	Once, the first time the servlet is requested (or at container startup, if load-on-startup is set)	One-time setup: open DB connections, read configuration, allocate shared resources.
service()	Once per incoming request (dispatches internally to doGet/doPost/doPut/... based on the HTTP method)	Handle the request: read parameters, run business logic, write the response.
destroy()	Once, when the container removes the servlet from service (shutdown, redeploy)	Cleanup: close connections, release resources, flush logs.

GET vs POST for sensitive data

POST is the appropriate method for handling sensitive registration data, and doPost() should perform the actual save:

- GET appends parameters to the URL (e.g. ?email=...®No=...), so sensitive values end up in browser history, server access logs, proxy logs, and can be bookmarked or shared accidentally.
- GET requests are also cached by browsers/proxies by default, which risks sensitive data being stored outside the application's control.
- URLs have practical length limits, which can silently truncate large forms; POST has no such restriction since data travels in the request body.
- POST is intended for state-changing operations (creating a student record) — this also matches REST/HTTP semantics, where GET should be safe and idempotent (read-only).

GET remains appropriate only for the non-sensitive step of simply displaying the empty form.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

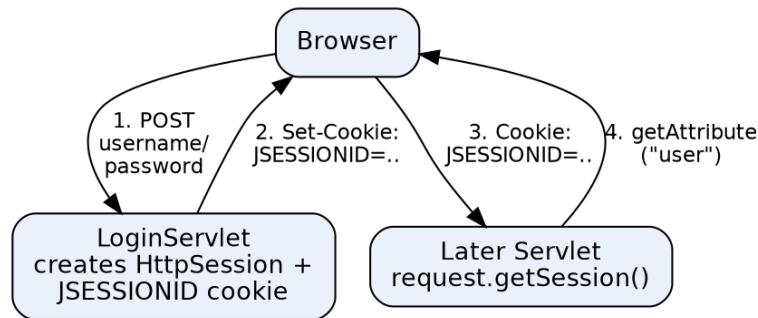
Answer:

How session tracking works

HTTP is stateless — each request is independent, so the server has no built-in memory of a previous request from the same user. Session tracking solves this by attaching a unique identifier to the user's browser, which is sent back on every subsequent request so the server can look up that user's stored state:

- On first contact, the container creates an HttpSession object and a unique session ID.

- The session ID is sent to the browser as a cookie named JSESSIONID (or appended to URLs via URL rewriting if cookies are disabled).
- On every later request, the browser automatically resends that cookie; the container uses it to fetch the matching HttpSession object and its stored attributes.



Login Servlet – creating a session

```

@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        if (AuthService.isValid(user, pass)) {
            HttpSession session = req.getSession(true); // create if absent
            session.setAttribute("username", user);
            session.setMaxInactiveInterval(30 * 60); // 30 minutes
            resp.sendRedirect("dashboard");
        } else {
            resp.sendRedirect("login.html?error=1");
        }
    }
}

```

Any Later Page – reading the session

```

@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        HttpSession session = req.getSession(false); // don't create a new one
        if (session == null || session.getAttribute("username") == null) {
            resp.sendRedirect("login.html"); // not authenticated
            return;
        }
        String user = (String) session.getAttribute("username");
    }
}

```

```

req.setAttribute("user", user);
req.getRequestDispatcher("dashboard.jsp").forward(req, resp);
}
}

```

Logout – invalidating the session

```

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        if (session != null) session.invalidate();
        resp.sendRedirect("login.html");
    }
}

```

Equivalent design using plain Cookies

Instead of relying on the container's HttpSession, the application could manage a token itself:

```

// After successful login:
Cookie authCookie = new Cookie("authToken", token);
authCookie.setHttpOnly(true); // not accessible to JavaScript
authCookie.setSecure(true);   // only sent over HTTPS
authCookie.setMaxAge(30 * 60); // 30 minutes
resp.addCookie(authCookie);

// On a later request:
Cookie[] cookies = req.getCookies();
String token = Arrays.stream(cookies)
    .filter(c -> c.getName().equals("authToken"))
    .map(Cookie::getValue).findFirst().orElse(null);
// token is then looked up in a server-side store (DB/cache) to find the user

```

HttpSession vs. Cookies

Aspect	HttpSession	Cookies
Where data lives	On the server; only an ID is sent to the client	Entirely on the client (browser)
Data types	Any Java object (setAttribute(String, Object))	Text only (name=value strings)
Size limit	Effectively bounded by server memory	~4 KB per cookie, limited cookies per domain
Security	Safer — sensitive data never leaves the server	Riskier if it holds sensitive data directly; needs HttpOnly/Secure flags

Dependency	Still needs a cookie (JSESSIONID) or URL rewriting to carry the session ID	Is itself the storage/transport mechanism
Lifetime control	Server-side timeout (setMaxInactiveInterval) or invalidate()	Client-side expiry (setMaxAge); user can also delete manually
Scalability	Needs sticky sessions or a shared session store across servers	Naturally stateless server-side — easier to scale horizontally

In practice, HttpSession is generally preferred for authentication state because sensitive data stays server-side, while cookies remain essential as the transport that carries the session identifier itself — the two techniques work together rather than being fully alternative choices.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

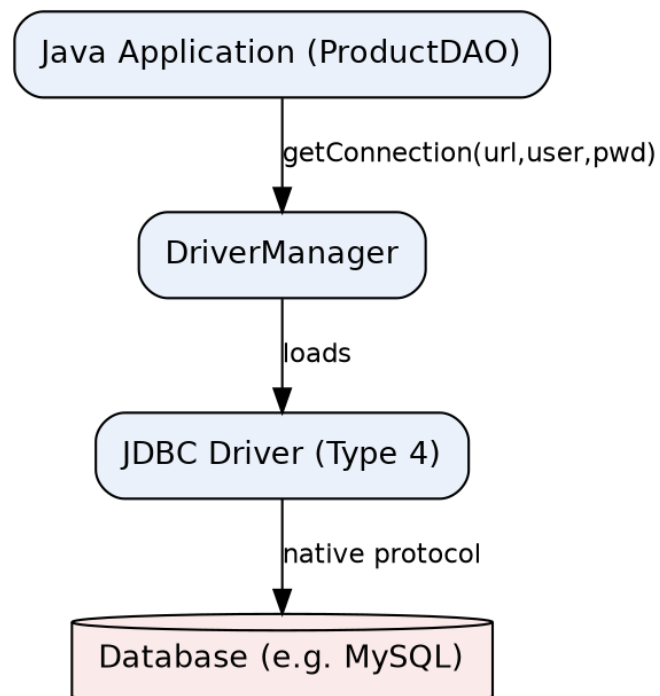
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer:

JDBC Architecture

JDBC (Java Database Connectivity) is a standard Java API that lets an application talk to a relational database independent of which vendor's database is used:

- JDBC API — the java.sql / javax.sql interfaces (Connection, Statement, ResultSet, etc.) that application code programs against.
- DriverManager — locates and loads the correct JDBC driver for a given connection URL.
- JDBC Driver — a vendor-specific implementation (typically a 'Type 4' pure-Java driver, e.g. MySQL Connector/J) that translates JDBC calls into the database's native network protocol.
- Database — the actual RDBMS (MySQL, PostgreSQL, Oracle, etc.) that executes the SQL and returns results.



Steps involved in database connectivity

- 1. Load/register the driver (modern JDBC 4+ drivers auto-register via META-INF/services, so this is usually implicit).
- 2. Open a connection with `DriverManager.getConnection(url, user, password)`.
- 3. Create a `Statement` or (preferably) a `PreparedStatement` to avoid SQL injection and benefit from query plan caching.
- 4. Execute the query: `executeUpdate()` for INSERT/UPDATE/DELETE, `executeQuery()` for SELECT (returns a `ResultSet`).
- 5. Process the `ResultSet` by iterating with `rs.next()` and reading columns.
- 6. Close all resources (`ResultSet`, `Statement`, `Connection`) — ideally with try-with-resources, so they close automatically even if an exception occurs.

ProductDAO – insert and retrieve

```
public class ProductDAO {
    private static final String URL =
        "jdbc:mysql://localhost:3306/shopdb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASS = "password";

    // ---- INSERT ----
    public void addProduct(String name, double price, int qty) {
        String sql = "INSERT INTO products(name, price, qty) VALUES (?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = conn.prepareStatement(sql)) {
```

```

        ps.setString(1, name);
        ps.setDouble(2, price);
        ps.setInt(3, qty);
        int rows = ps.executeUpdate();
        System.out.println(rows + " row(s) inserted.");

    } catch (SQLException e) {
        System.err.println("Insert failed: " + e.getMessage());
        // In a real app: log via a logging framework and/or
        // rethrow as a custom DataAccessException.
    }
}

// ---- RETRIEVE ----
public List<Product> getAllProducts() {
    List<Product> list = new ArrayList<>();
    String sql = "SELECT id, name, price, qty FROM products";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            list.add(new Product(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getDouble("price"),
                rs.getInt("qty")));
        }

    } catch (SQLException e) {
        System.err.println("Query failed: " + e.getMessage());
    }
    return list;
}

```

Exception handling notes

- try-with-resources (used above) guarantees Connection, PreparedStatement, and ResultSet are closed in reverse order automatically, even if executeUpdate()/executeQuery() throws — avoiding leaked DB connections.
- SQLException carries a vendor error code and SQLState; catch it specifically rather than a generic Exception, and log e.getMessage() (and e.getSQLState() if useful) instead of exposing raw DB errors to end users.

- PreparedStatement with '?' placeholders (rather than string-concatenated SQL) prevents SQL injection, which matters directly for a public-facing e-commerce form.
- For multi-step operations (e.g. deduct stock and record an order together), wrap them in a transaction: `conn.setAutoCommit(false)`, then `conn.commit()` on success or `conn.rollback()` in a catch block.

Code of Conduct:

I Y.Sai Prasanth Varma (192411191) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y.Prasanth

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand